Implementing Hybrid Resource Analysis in Resource Aware ML 2

Arnav Sabharwal, Carnegie Mellon University

1 Introduction

Given a program P, the goal of resource analysis is to automatically derive a bound $f(x)$ on the worst-case resource use (e.g., time, space, stack depth) of P in terms of its input x. We say that $f(x)$ is *sound* if for all possible arguments x, $f(x)$ is a valid upper bound on the worst-case cost of $P(x)$. Deriving sound cost bounds has a number of applications in job scheduling, preventing side-channel attacks, etc.

There are two main classes of resource analysis techniques. Firstly, we have *static resource analyses*, which analyze the program's source code to reason about all possible behaviors. Their strength is that the bounds they derive are necessarily sound. However, due to the undecidability of resource analysis for Turing-complete languages, they are *incomplete*: there exist programs which cannot be analyzed tightly (or at all) by static resource analyses. Conversely, we have *data-driven analyses*, which proceed by logging the program's worst-case cost when run on a finite set of inputs chosen according to some distribution. Then, a cost bound is statistically inferred using the collected data. Data-driven analyses are able to derive bounds on programs which cannot be analyzed statically. However, since the set of program inputs may not induce worst-case behavior, there is no guarantee that statistically inferred bounds are sound.

Hybrid resource analyses [12, 13] integrate the completementary strengths and weaknesses of static and data-driven analyses. Informally, they allow the user to "delegate" whatever cannot be analyzed by purely static analysis to data driven analysis. Then, the static and data-driven bounds are composed in a principled manner to yield a final bound which (i) is closer to the ground truth than analyzing the entire program using a data-driven analysis and (ii) enables resource analysis for programs which cannot be analyzed tightly (or at all) by purely static methods. In this work, I implement and improve the hybrid resource analysis technique of *resource decomposition* [12] in Resource Aware ML (RaML) 2 [3, 4], a resource analysis tool targeting Standard ML (SML) programs. I motivate the need for resource decomposition in Section 2, describe its framework in Section 3, explain my implementation of it in RaML 2 in Section 4, discuss RaML 2 with resource decomposition in Section 5, and give an example of its use in Section 6.

2 Problem and Motivation

Automatic Amortized Resource Analysis (AARA) [7–10] is a technique for statically deriving *sound, worst-case, polynomial* cost bounds on functional programs. Here, the notion of cost is specified by the user by annotating the program with the $\mathrm{tick}(q), q \in \mathbb{Q}$ effect, as shown in Listing 1. When $q > 0$, the resource being specified is consumed, and when $q < 0$, it is freed up. The ability to free up resources means AARA is able to work with *non-monotone* notions of cost (e.g., memory allocations, recursion depth, etc.). We assume a by-value, *resource-aware* operational semantics which equips the normal evaluation judgment with the program's peak (high-water mark) and net cost. It becomes necessary to differentiate between the two in the presence of non-monotonicity. For a more formal treatment of the semantics, see [6].

Listing 1: Use of tick to specify number of function calls as notion of cost in mergesort

```
fun split l = tick 1.0; ...
fun merge l1 l2 = tick 1.0; ...
fun sort l =
  tick 1.0;
  case l of [] | [_] => l
  | _ :: _ :: _ =>
    let l1, l2 = split l in
    merge (sort l1) (sort l2)
```

Taking quicksort (always choosing list head as pivot) as an example, AARA is able to infer a sound quadratic worst-case bound of $1+2.5n+0.5n^2$ on the number of function calls, where n is the length of the input list. Conversely, since the average case complexity of quicksort is $O(n \log n)$, a data-driven analysis relying on randomly generated inputs is likely to infer an asymptotically incorrect $O(n \log n)$ cost bound on quicksort. In general, AARA yields tight cost bounds when the recursion scheme that induces worst-case behavior is structural.

However, AARA is currently unable to statically infer logarithmic cost bounds, which poses problems for efficient divide-and-conquer style programs. Taking mergesort in Listing 1 as an example, AARA infers an asymptotically loose $1 - 1.5n + 2.5n^2$ bound on number of function calls, due to its inability to tightly bound mergesort's recursion depth. In fact, for automated cost analysis tools that commit to soundness, correctly inferring logarithmic worst-case cost bounds remains an open problem.

3 Resource Decomposition

Resource decomposition [12] is a technique for integrating data-driven resource analyses into AARA, enabling cost analysis for programs that cannot be analyzed tightly. Given a program $P(x)$, it works as follows:

(1) The user annotates $P(x)$ with a new cost annotation primitive $\mathrm{mark}_l(z)$ $(z \in \mathbb{Z})$ used identically as $\mathrm{tick}(q)$ to specify a quantitative semantic property l that cannot be tightly bound by AARA (e.g., the recursion depth of a helper function within P). The semantic property tracked is called a *resource component*, and the annotated program is termed the *resource decomposed* program. Thus, Listing 2 is the resource decomposed version of Listing 1 where the resource component rd is the recursion depth of `sort`. Here, we extend the resource-aware semantics of [6] to (additionally) track the peak and net cost of each resource component l defined by $\mathtt{mark}_l$.

Listing 2: Resource decomposed version of mergesort, resource component specified is the recursion depth of sort

```
fun sort l =
```

```
  tick 1.0; mark_rd(1);
  let val res =
    case l of [] | [_] => l
    | _ :: _ :: _ =>
      let l1, l2 = split l in
      merge (sort l1) (sort l2)
  in mark_rd(-1); res
```

(2) A data-driven analysis is used to derive a bound $g(x)$ on the peak value of l. In the case of Listing 2, we obtain a concrete logarithmic bound on the peak value of the resource component *rd* tracking the recursion depth of *sort*. Meanwhile, the resource decomposed program $P(x)$ is transformed to be parameterized over an additional variable r, which tracks the resource component l specified by the user. The resultant program $P(x, r)$ is termed the *resource guarded program*.

Listing 3: Resource guarded version of mergesort

```
fun sort l rd =
  case rd of
  Zero => raise exception
  | Succ rd' =>
    tick 1.0;
    case l of [] | [_] => l
    | _ :: _ :: _ =>
      let l1, l2 = split l in
      let l1_s = sort l1 rd' in
      let l2_s = sort l2 rd' in
      merge l1_s l2_s
```

For example, in Listing 3, sort l is given an additional ghost argument [11] *rd*, denoting the maximum recursion depth of *sort* for sorting any list of length $|l|$. Note that *rd* being 0 would imply a contradictory state of affairs, since no list can be sorted by *sort* with 0 recursion depth (hence the exception). The rest of the definition is forced to maintain the meaning of the ghost argument. Note also how the changes to *rd* are dictated exactly by the user's $\text{mark}_{\text{rd}}(-)$ annotations.

(3) Finally, AARA is used to derive a sound worst-case bound $f(x, r)$ on the resource guarded program. Composing this with our data-driven bound, we report $f(x, g(x))$ as the final cost bound for $P(x)$. For example, AARA infers a bound of $1 + 3.5 * |l| * \text{rd}$ on Listing 3. If data-driven analysis reports that $\text{rd} \leq \log_2(|l|) + 2$, then the final bound on the number of function calls made by mergesort is $1 + 3.5 * |l| * (2 + \log_2 |l|)$.

4 Implementation

My contributions pick up from where [12] left off towards integrating resource decomposition into RaML 2. I provide a bird's eye view of the architecture in Figure 1. The first phase, which is not my contribution, converts resource decomposed SML source programs into a simplified IR, which we call TML.

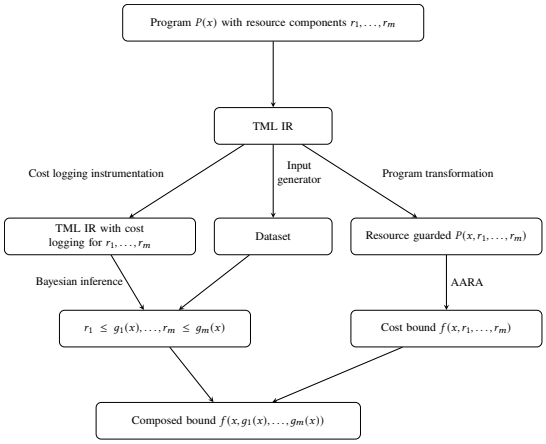

Figure 1: Architecture for the inclusion of resource decomposition in RaML 2

4.1 Program transformation from resource decomposed to resource guarded programs

Given a TML program $P(x)$ that treats the recursion depths of helper functions $f_1, \ldots, f_m$ as resource components, I implemented a transformation that automatically converts $P(x)$ into its resource-guarded version $P(x, r_1, \ldots, r_m)$, where $r_i : \mathbb{N}$ is a ghost variable representing the worst-case recursion depth of f_i. Previously [12], this was done by hand. I consider $\text{mark}_l(z)$ as a resource effect, and expressions that may use it as resource effectful. TML does not make a type distinction for resource effectful expressions, since the syntactic form for $\text{mark}_l(z)$ is typed as:

$$\frac{z \in \mathbb{Z} \qquad \Gamma \vdash e : \rho}{\Gamma \vdash \text{mark}_l(z)\ ;\ e : \rho}\ \textsc{mark}$$

Thus, the transformation proceeds by introducing the monadic type $F(A)$, denoting resource-effectful computations returning a value $v : A$. Then, the resource decomposed program is monadified such that all resource-effectful expressions returning value $v : A$ have the type $F(A)$. So, the updated MARK rule looks like:

$$\frac{z \in \mathbb{Z} \qquad \Gamma \vdash e : F(\rho)}{\Gamma \vdash \text{mark}_l(z)\ ;\ e : F(\rho)}\ \textsc{markf}$$

Having introduced a type distinction for resource effectful expressions, we may produce our resource guarded TML program using a type directed translation derived from [12]. Assuming P contains exactly m resource components enumerated as $1 \ldots m$, the key rule in the translation of types is:

$$\|F(A)\| = \mathbb{N}^m \to \mathbb{N}^m \times \|A\|$$

That is, resource effectful expressions are to be understood as functions taking the "remaining" amount of each resource component $\langle r_1, \ldots, r_m \rangle : \mathbb{N}^m$ to a pair containing the updated resource state $\langle r'_1, \ldots, r'_m \rangle : \mathbb{N}^m$ and the (transformed) result $v : \|A\|$. For example, the translation for MARKF is:

$$\|\text{mark}_l(z)\,;e\| = \langle r_1, \ldots, r_m \rangle \mapsto \begin{cases} \|e\| \langle \ldots, (r_l - z), \ldots \rangle & r_l \geq z \\ \text{raise exception} & \text{otherwise} \end{cases}$$

If z is positive, then the remaining resources for component l are decremented by z. Since we work under the assumption that r_l is to be instantiated at the toplevel with the peak cost on resource component l, if $r_l < z$, we raise an exception to signify that we are in contradictory state of affairs. Assuming that the program P is a function with an effectful body, my transformation converts $P(x)$ into the resource guarded program $P(x, r_1, \ldots, r_m)$ which is free of all `mark` annotations. This can now be analyzed by RaML 2's own implementation of AARA to produce a sound, worst-case cost bound in terms of the original input size x and the newly introduced ghost variables $r_1, \ldots, r_m$. For example, since mergesort treats its own recursion depth as a resource component, its type in the resource-guarded program becomes $\text{list}(\mathbb{Z}) \to \mathbb{N} \to \text{list}(\mathbb{Z})$. Thus, given Listing 2, the automated translation yields Listing 3, albeit Listing 3 is simplified using the fact that the recursion depth of any function is zero before and after its toplevel call is entered and exited respectively.

4.2 Data-driven analysis

I extended the data driven analysis pipeline of [12] to be able to handle programs $P(x)$ where x can be of arbitrary input type τ. To this end, I created a random input generator for arbitrary monomorphic and polynomial SML datatypes. The first phase of the data-driven analysis uses this generator to create a set X of uniformly random values of type α given the resource decomposed program $P(x : \alpha)$.

Independently, the user is expected to define a multidimensional size function $s : \tau \to \mathbb{N}^k$, where the number of size dimensions k is chosen by the user. For example, if $\tau = \text{list}(\text{list}(\text{unit}))$, then a candidate size function is $s([l_1, \ldots, l_m]) = (m, \sum_{i=1}^{m} |l_i|)$. Using the generated set of program inputs X, for each $x \in X$, I run $P(x)$ against an SML structure that records the peak recursion depth $d_i(x)$ attained by each of $f_1, \ldots, f_m$ at runtime. For each f_i, the $(s(x), d_i(x))$ tuples form our runtime cost dataset $\mathcal{D}_i$.

For each $\mathcal{D}_i$, I use mutual information as in [12] to find the component of the user-defined size function's output that the observed recursion depth most depends on. This induces the one-dimensional size function $s' : \tau \to \mathbb{N}$ which, given a $v : \tau$, chooses the most dependent component of $s(v)$, and the simplified dataset $\mathcal{D}'_i$ containing $(s'(x), d_i(x))$ for each $x \in X$. Finally, by implementing the Bayesian inference model specified in [12] in STAN [2] (a probabilistic programming language), I am able to infer a concrete, univariate linear or logarithmic recursion depth bound on each of $f_1, \ldots, f_m$.

5 Discussion and Comparison with Prior Work

Let the program $P(x)$ use the function $f : A \to B$ as a helper, such that the user wishes to treat the worst-case recursion depth of f as a resource component. The original formulation of resource decomposition derives a bound $g(x)$ on f's recursion depth in terms of the size of x, the *global* argument. This subtly introduces a dependence on data-driven analysis to bound the size of the argument to f in terms of x (for simplicity, assume that f has only 1 callsite outside its definition). This is unwanted, since we should only depend on data-driven methods for recursion depth bounding, which cannot be handled by AARA alone. My current implementation breaks down the process of inferring $g(x)$ into 2 independent steps:

(1) Given $f(y : A)$, we first derive a bound on f's recursion depth in terms of the size of its own argument $y : A$. Call this $g'(y)$.
(2) Then, using a static analysis, I derive a sound worst-case bound $s(x)$ on the size of y in terms of the global input x. Finally, we may compose the two bounds to report $g'(s(x))$ as the worst-case recursion depth of f in terms of the size of x. This size-bounding phase is currently work in progress, and must be performed manually for the moment.

Another task the user needs to do manually is the definition of the size functions in Section 4.2, which can involve choice for types admitting multiple reasonable notions of size. Whilst the user currently needs to annotate functions with `mark` to track their recursion depth, this process is mechanical and can be automated. Thus, barring the aforementioned manual tasks, my contributions, to a large extent, automate the resource decomposition technique formulated in [12]. Finally, since I implement a generator that doesn't take into account the code for the program P, there is no guarantee that the set of randomly generated inputs induces worst-case recursion depth behavior. So, the inclusion of data-driven methods undoubtedly introduces unsoundness in our resource analysis. We specifically supplement AARA with data-driven analysis since in comparison to other static analyses based on sized types [14], recurrence relations [5], and defunctionalization [1], AARA supports all of (i) higher order functions, (ii) abstract notions of cost and (iii) derivation of amortized cost bounds.

6 Example of Use

Consider a program $\text{bst} : \text{list}(\mathbb{Z}) * \text{list}(\mathbb{Z}) \to \text{unit}$, such that for $\text{bst}(l_1, l_2)$, we first run mergesort on l_1, create a balanced tree t using it, and then lookup each element in l_2 in the tree t. Assuming our notion of cost is the number of function calls, the ground truth asymptotic worst-case bound is $O((|l_1| + |l_2|) * \log_2 |l_1|)$. AARA yields a bound of $3 - 0.5|l_1| + |l_1| * |l_2| + 2.5|l_1|^2 + 2|l_2|$ on bst. The $|l_1|^2$ term indicates an inability to tightly bound the recursion depth of mergesort, and the $|l_1| * |l_2|$ term indicates an inability to tightly bound the recursion depth of lookups in a balanced BST. Thus, the recursion depths of mergesort and lookup form our resource components.

Considering d_m and d_l to be the worst-case recursion depths of mergesort and BST lookup respectively, the bound I derive on the resource-guarded version of bst is $3 + |l_2| * d_l + 2.5|l_1| * d_m + |l_2|$. Independently, by running bst on 200 (number chosen arbitrarily) randomly generated values of type $\text{list}(\mathbb{Z}) * \text{list}(\mathbb{Z})$, I derive the following data-driven bounds:

(1) $d_l \leq 0.6 + 1.5 \log_2(0.6 + 1.8|t|)$, where $|t|$ is the number of nodes in the balanced BST used by the lookup function
(2) $d_m \leq 1.3 + 1.5 \log_2(0.6 + 2.4|l_1|)$

Now, to derive a final, tight bound in terms of $|l_1|$ and $|l_2|$, it suffices to express $|t|$ in terms of $|l_1|$. While we know $|t| = |l_1|$ in this case, I'm currently exploring methods to automate this phase of the analysis for arbitrary programs.

Acknowledgments

I would like to thank my advisor, Jan Hoffmann and his students Long Pham (former) and Ethan Chu for their guidance and support.

References

[1] Avanzini, M., Dal Lago, U., and Moser, G. Analysing the complexity of functional programs: higher-order meets first-order. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming* (2015), pp. 152–164.

[2] Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., Brubaker, M., Guo, J., Li, P., and Riddell, A. Stan: A probabilistic programming language. *Journal of Statistical Software 76*, 1 (2017), 1–32.

[3] Chu, E. The theory and implementation of resource aware ml 2. Master's thesis, Carnegie Mellon University, Pittsburgh, PA, December 2023. Available at https://www.andrew.cmu.edu/user/ethanchu/raml-thesis.pdf.

[4] Chu, E., Guo, Y., and Hoffmann, J. Handling exceptions and effects with automatic resource analysis. *OOPSLA 10*, OOPSLA1 (Apr. 2026).

[5] Danner, N., Licata, D. R., and Ramyaa, R. Denotational cost semantics for functional languages with inductive types. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming* (2015), pp. 140–151.

[6] Hoffmann, J. Types with potential: polynomial resource bounds via automatic amortized analysis, October 2011.

[7] Hoffmann, J., Aehlig, K., and Hofmann, M. Multivariate amortized resource analysis. *ACM Trans. Program. Lang. Syst. 34* (2012), 14:1–14:62.

[8] Hoffmann, J., and Hofmann, M. Amortized resource analysis with polynomial potential. In *European Symposium on Programming* (2010).

[9] Hoffmann, J., and Jost, S. Two decades of automatic amortized resource analysis. *Mathematical Structures in Computer Science 32* (2022), 729 – 759.

[10] Hofmann, M., and Jost, S. Static prediction of heap space usage for first-order functional programs. In *ACM-SIGACT Symposium on Principles of Programming Languages* (2003).

[11] Hofmann, M., and Pavlova, M. Elimination of ghost variables in program logics. In *Proceedings of the 3rd Conference on Trustworthy Global Computing* (Berlin, Heidelberg, 2007), TGC'07, Springer-Verlag, p. 1–20.

[12] Pham, L., Niu, Y., Glover, N., Saad, F., and Hoffmann, J. Integrating resource analyses via resource decomposition. *Proc. ACM Program. Lang. 9*, OOPSLA2 (Oct. 2025).

[13] Pham, L., Saad, F. A., and Hoffmann, J. Robust resource bounds with static analysis and bayesian inference. *Proceedings of the ACM on Programming Languages 8*, PLDI (2024), 76–101.

[14] Vasconcelos, P. B. *Space cost analysis using sized types*. PhD thesis, University of St Andrews, 2008.